

UNIVERSITÉ DE SHERBROOKE · ÉCOLE DE GESTION

GIS805

Session 08 / 12

Ponts pondérés, SCD avancés et relations many-to-many chez NexaMart

Structuration et analyse multidimensionnelle



PROFESSEUR

**Prof. Carlos Denner
dos Santos**

Département des systèmes
d'information et méthodes
quantitatives

DATE

8 juin 2026

LIEU

Longueuil

HORAIRE

19 h 00 – 22 h 00

Votre séance, en bref

Voici comment se déroule votre soirée — 3 temps forts.

20 min

Bridge theory + SCD3 intro

45 min

Sprint 1 : weighted bridge

40 min

Sprint 2 : reconciliation + campaign allocation



*Comment allouer les couts et
comprendre les segments clients qui
se chevauchent sans double-
compter ?*

— Le CEO de NexaMart Group, séance 8

Le problème : un client, plusieurs segments

Les relations many-to-many cassent les agregations

- Un client NexaMart est à la fois Gold ET "Acheteur frequent" ET "Early adopter"
- **Si on joint directement** : le revenue de ce client est triple
- Le CEO voit 3x le chiffre réel -- c'est un mensonge
- **Solution de Kimball** : la table pont (bridge table) avec poids

Le schema de ce soir

Comment le pont connecte vos tables existantes

- fact_sales → dim_customer (customer_key) -- deja en place depuis S02
- dim_customer → bridge_customer_segment (customer_key) -- NOUVEAU
- bridge_customer_segment → dim_segment_outtrigger (segment_key) -- NOUVEAU
- Le pont est entre dim_customer et dim_segment_outtrigger
- **Grain du pont** : une ligne = un client x un segment x un poids
- Grain de fact_sales est inchange -- le pont ne touche pas la table de faits

❶ Consultez docs/visuals/bridge-m2n.md pour le diagramme ER complet avec Mermaid.

Avant de commencer : votre inventaire

DEJA FAIT PAR LE PIPELINE

- `sql/bridges/bridge_customer_segment.sql`
-- cree la table pont
- `sql/dims/dim_customer_scd3.sql` -- cree la dim SCD Type 3
- `sql/dims/dim_segment_outtrigger.sql` -- cree la dim outtrigger
- `scripts/datagen/gen_s08_bridges.py` -- genere les CSVs
- `make generate && make load` -- tout se construit

A CREER PAR VOUS CE SOIR

- `sql/bridges/s08-weighted-allocation.sql` -- votre requete ponderee
- `sql/checks/s08-weighted-reconciliation.sql` -- vos checks
- `answers/S08_executive_brief.md` -- votre brief au CEO
- Lisez le SQL existant AVANT d ecrire le votre
- Consultez `docs/worked-examples/s08-bridge-returns-walkthrough.md`

01 Le pont pondere

Resoudre le many-to-many sans double-comptage

Anatomie d'un pont pondere

bridge_customer_segment

- **customer_key** : le client
- **segment_key** : le segment
- **weight** : la fraction du revenue attribuee a ce segment
- **Contrainte** : $\text{SUM}(\text{weight}) = 1.0$ pour chaque customer_key
- **Exemple** : Gold = 0.5, Frequent = 0.3, Early = 0.2

Comment utiliser le pont en requête

Revenue par segment sans duplication

- `fact_sales JOIN bridge_customer_segment ON customer_key`
- `revenue_pondere = revenue * weight`
- `SUM(revenue_pondere) GROUP BY segment`
- Le total pondere doit éгалer le total réel -- c'est la vérification obligatoire
- Si `SUM(weight) != 1.0` pour un client, le pont est casse

Sans pont vs avec pont

JOIN DIRECT (ERREUR)

- Client dans 3 segments = 3x le revenue
- Total gonfle de 200%
- Le CEO prend une mauvaise décision
- Impossible à détecter sans vérification

PONT PONDERE (CORRECT)

- Client dans 3 segments = revenue reparti
- Total pondere = total réel
- Le CEO voit la repartition réelle
- Vérification : $\text{SUM}(\text{weight}) = 1.0$ par client

FAUX : JOIN direct sans pondération

Le double-comptage en action -- exécutez ceci et observez

```
-- ❌ FAUX : jointure directe client → segments (pas de poids)
SELECT seg.segment,
       SUM(f.line_total) AS revenue
FROM   fact_sales f
       JOIN dim_customer c          ON c.customer_key = f.customer_key
       JOIN bridge_customer_segment b ON b.customer_key = c.customer_key
       JOIN dim_segment_outtrigger seg ON seg.segment_key = b.segment_key
GROUP BY 1;

-- Marie est Gold + Fidèle + Web → ses 100 $ comptent 3 fois
-- SUM(revenue) par segment = 3 × le vrai total → MENSONGE

-- Vérification immédiate :
SELECT SUM(line_total) AS vrai_total FROM fact_sales;
-- Comparez avec la somme des segments ci-dessus : si > vrai_total → double-comptage
```

❗ Si la somme des segments dépasse le total réel, vous avez un fan-out. Exercice 1 vous fera vivre ce problème.

CORRECT : revenue pondere via le pont

Chaque vente est repartie proportionnellement aux segments

```
-- ✓ CORRECT : multiplier par le poids du pont
SELECT seg.segment,
       SUM(f.line_total * b.weight) AS revenue_alloue
FROM   fact_sales f
       JOIN dim_customer c          ON c.customer_key = f.customer_key
       JOIN bridge_customer_segment b ON b.customer_key = c.customer_key
       JOIN dim_segment_outtrigger seg ON seg.segment_key = b.segment_key
GROUP BY 1
ORDER BY 2 DESC;

-- Marie (100 $) : Gold = 50 $, Fidèle = 30 $, Web = 20 $
-- Total = 100 $ = vrai total ✓
-- La clé : SUM(weight) = 1.0 par customer_key garantit la réconciliation
```

❗ Essayez d'abord en Exercice 1 avant de consulter ce slide. La seule difference avec le FAUX : « * b.weight ».

Reconciliation obligatoire : total pondere = total reel

Ce check est un livrable -- pas un bonus

```
-- Check 1 : SUM(weight) = 1.0 par client
SELECT customer_key,
       SUM(weight)      AS total_weight,
       CASE WHEN ABS(SUM(weight) - 1.0) > 0.01
            THEN 'FAIL' ELSE 'PASS' END AS result
FROM   bridge_customer_segment
GROUP BY customer_key
HAVING ABS(SUM(weight) - 1.0) > 0.01;
-- Resultat attendu : 0 lignes. Si des lignes apparaissent → pont cassé.

-- Check 2 : total pondéré = total réel
SELECT
  'total_sans_pont' AS methode, SUM(f.line_total) AS total
FROM fact_sales f
UNION ALL
```

❗ Sauvegardez ces checks dans `sql/checks/s08-weighted-reconciliation.sql`. Ils seront évalés.

02

SCD Type 3 : current vs previous

Quand on veut comparer avant/apres sans historique complet

SCD Type 3 en pratique

Deux colonnes au lieu de deux lignes

- **dim_customer** : segment_current = Platinum, segment_previous = Gold
- Une seule ligne par client -- pas de gonflement de table
- **Permet** : "combien de clients sont passes de Gold a Platinum ?"
- **Limite** : ne garde qu'un seul changement
- **Combine naturellement avec le pont** : le pont alloue, le SCD3 compare

SCD Type 3 : implementation dans dim_customer

Deux colonnes dans la même ligne -- pas de nouvelle ligne

```
-- Créer dim_customer_scd3 depuis les données S08
CREATE OR REPLACE TABLE dim_customer_scd3 AS
SELECT
    c.customer_key,
    c.customer_id,
    c.full_name,
    h.current_segment,
    h.previous_segment,          -- NULL si jamais changé
    CAST(h.segment_change_date AS DATE) AS segment_change_date,
    h.city,
    h.province
FROM dim_customer c
    JOIN raw_customer_scd3_history h ON h.customer_id = c.customer_id
WHERE c.is_current = TRUE;
```

i SCD3 est le concept bonus de ce soir. Implementez-le seulement apres avoir termine le pont et les verifications.

Pont campagne : meme principe, autre contexte

raw_bridge_campaign_allocation : budget reparti par segment

- Une campagne marketing cible plusieurs segments simultanement
- **Chaque segment recoit une fraction du budget** : budget_weight
- $\text{cout_alloue} = \text{planned_spend} * \text{budget_weight}$
- **Meme contrainte** : $\text{SUM}(\text{budget_weight}) = 1.0$ par campagne
- **Meme verification** : total alloue = total prevu
- Vous ecrirez cette requete en Sprint 2 -- memes reflexes que le pont client

❗ raw_bridge_campaign_allocation est une table brute (pas transformee par le pipeline). Vous la requetez directement.

Votre copilote cette semaine

Comment travailler avec votre assistant IA pour les ponts pondérés

✓ PROMPTS QUI MARCHENT

- Pourquoi un client dans 2 segments cause du double-comptage sans pont pondéré ?
- Génère `bridge_customer_segment` avec la contrainte $\text{SUM}(\text{weight})=1.0$ par client.
- Écris la requête de réconciliation : total pondéré = total réel.

✗ PROMPTS À REJETER

- l'assistant attribue des poids égaux à tous les clients multi-segments
- le total pondéré ne réconcilie pas avec le total réel

03

Sprint 1 : comprendre et utiliser le pont pondere

Lisez le SQL du pont, puis ecrivez vos requetes d'allocation

Etape 0 : charger les donnees S08

Executez ces commandes AVANT de commencer les exercices

```
# Terminal : generer les CSVs et charger la base
make generate && make load

# Resultat attendu :
#   raw_bridge_customer_segment      288 rows
#   raw_bridge_campaign_allocation    17 rows
#   raw_customer_scd3_history         189 rows
#   raw_dim_segment_outtrigger        6 rows
#   bridge_customer_segment           ~428 rows (cree par sql/bridges/)
#   dim_segment_outtrigger            6 rows (cree par sql/dims/)
#   dim_customer_scd3                189 rows (cree par sql/dims/)

# Verifier :
make check
# 31 PASS 0 FAIL 1 inqlant BRIDGE WEIGHT SUM(weight)=1 0 PASS
```

❗ Si make load echoue, verifiez que sql/bridges/ est un DOSSIER contenant bridge_customer_segment.sql (pas un fichier nomme bridges).

Exercice 0 : lisez le SQL du pont

Ouvrez `sql/bridges/bridge_customer_segment.sql` dans votre editeur

- **Ligne 1** : `FROM raw_bridge_customer_segment` -- d ou viennent les poids ?
- **Ligne 2** : `JOIN dim_customer ON customer_id` -- pourquoi ? (obtenir `customer_key`)
- **Ligne 3** : `JOIN dim_segment_outtrigger ON segment` -- pourquoi ? (obtenir `segment_key`)
- Le pont couvre TOUTES les versions SCD2 du client -- pourquoi ?
 - → Parce que `fact_sales` a des ventes liees a des `customer_key` historiques
- **Le check a la fin** : `SUM(weight) = 1.0` par `customer_key`
- **Après lecture** : explorez la table avec `DESCRIBE` et `SELECT COUNT(*)`

❗ C est l etape la plus importante de ce soir. Si vous ne comprenez pas ce SQL, vous ne pouvez pas ecrire le votre.

Sprint 1 : votre mission

Fichier cible : `sql/bridges/s08-weighted-allocation.sql`

- **Etape 0** : make generate && make load. Le pont `bridge_customer_segment` est cree automatiquement.
- **Etape 1 (Exercice 0)** : ouvrez `sql/bridges/bridge_customer_segment.sql` et LISEZ le SQL. Repondez aux questions du guide.
 - → Comprenez chaque jointure avant d'ecrire votre propre requete.
- **Etape 2 (piege)** : ecrivez la requete naive `SUM(line_total) GROUP BY segment` SANS multiplier par `weight`.
 - → Comparez avec `SELECT SUM(line_total) FROM fact_sales`. Notez le ratio d'inflation (~1.5x).
- **Etape 3** : ecrivez la requete correcte avec `SUM(line_total * b.weight) AS revenue_alloue`.
- **Etape 4** : verifiez `SUM(weight) = 1.0` par client. 0 lignes = OK.
- **Etape 5** : reconciliation : `total_sans_pont = total_avec_pont (ecart = 0.00)`.

❗ Le pont est de l'infrastructure (deja fait). Votre livrable est la requete d'analyse et la reconciliation.

Sprint 1 : votre mission (suite)

- `git add -A && git commit -m 'S08 sprint1 weighted allocation + reconciliation'`

04

Sprint 2 : reconciliation + allocation campagne

Prouvez que vos totaux sont corrects

Le board brief : ce que le CEO attend

answers/S08_executive_brief.md -- structure recommandee

- **1. La question** : "Comment repartir le revenu entre segments qui se chevauchent ?"
- **2. La methode** : pont pondere, contrainte $\text{SUM}(\text{weight}) = 1.0$
- **3. Les chiffres** : tableau comparatif attribution primaire vs ponderee par segment
- **4. L observation** : quel segment est surestime sans ponderation ? Sous-estime ?
- **5. La recommandation** : quelle methode pour quels rapports ? Pourquoi ?
- Appuyez chaque affirmation avec un chiffre du pont -- pas d opinions sans preuve

❗ Consultez content/courses/GIS805/reference-solution/answers/S08_executive_brief.md pour un exemple (apres avoir essaye).

Sprint 2 : votre mission

Fichiers cibles : `sql/checks/s08-weighted-reconciliation.sql` + `answers/S08_executive_brief.md`

- **Etape 0** : sauvegardez vos checks de reconciliation dans `sql/checks/s08-weighted-reconciliation.sql`.
- **Etape 1 (SCD3 -- bonus)** : ouvrez `sql/dims/dim_customer_scd3.sql`, lisez le SQL. La table existe deja.
 - → Explorez : combien de clients ont change de segment ? Quelles transitions sont les plus frequentes ?
- **Etape 2** : allocation campagne -- ecrivez la requete de cout alloue par segment via `raw_bridge_campaign_allocation`.
 - → Memes principes : `SUM(planned_spend * budget_weight)`, reconciliation obligatoire.
- **Etape 3** : board brief dans `answers/S08_executive_brief.md`.
 - → Repondez a la question du CEO avec evidence provenant du pont pondere.
 - → Montrez la difference : attribution primaire vs attribution ponderee.
- `git add -A && git commit -m 'S08 sprint2 SCD3 + campaign + brief'`

❗ Le SCD3 est un concept bonus. Ne le forcez que si vous avez termine le pont et les verifications.

Soumission et evaluation

Ce qui est attendu avant S09

- **Fichiers** : answers/S08_executive_brief.md + sql/bridges/s08-weighted-allocation.sql
- **Vérification** : sql/checks/s08-weighted-reconciliation.sql
- **Evaluation** : modèle (40%), validation (25%), justification exécutive (20%)
- Trace de processus (10%), reproductibilité (5%)
- **Commitez** : git add -A && git commit -m "S08 bridges" && git push
- Consultez docs/verify-before-pushing.md avant de pousser

Où en sommes-nous ?

Votre parcours en 14 séances

- S01-S07 : Etoile, grain, SCD, dims spéciales, integration, drill-across [FAIT]
- >> **S08** : Ponts ponderes, many-to-many [AUJOURD'HUI]
- S09 : 4 types de tables de faits
- S10 : Examen intra 2 (S06-S09)
- S11-S14 : Documentation, défense, au-dela, final

A retenir de ce soir

- 1 Aucun pont n'est accepte sans preuve que totaux ponderes = totaux réels.
- 2 SUM(weight) = 1.0 par entite est la contrainte fondamentale du pont.
- 3 SCD Type 3 = current + previous dans la même ligne, sans historique complet.
- 4 La vérification est un livrable, pas un bonus.
- 5 La semaine prochaine : les 4 types de tables de faits (S09).